学案 01 | 模型、文本与训练/推理

1. 先从一个不神秘的任务开始

假设模型看到一句话：“今天外面下雨，我出门要带一把……”它需要根据前文给出后续内容的可能性。模型内部不是先写好整句答案再拿出来，而是根据已给内容计算下一个单位的候选分数，再按生成规则选出一个单位，接着重复。

这里的“单位”通常不是一个汉字，也不保证是一个完整单词，而是 tokenizer 产生的 **token**。Tokenizer 把文本拆成 token，再把每个 token 映射成整数编号 **token ID**。模型主要处理这些编号及其对应的数值表示，不直接处理屏幕上看到的字符。

本阶段只记住概念，不要求会设计 tokenizer。实际模型会使用特定词表和切分规则，同一句话在不同 tokenizer 下可能对应不同的 token 序列。

2. 一条最小处理流程

```
人类可读文本
    ↓ tokenizer
token 序列
    ↓ 转成编号
token ID 序列
    ↓ 模型计算（由参数决定）
候选输出的分数/概率
    ↓ 训练时和正确目标比较；生成时选择一个 token
损失信号 或 新生成 token
```

参数（parameters） 是模型从数据中学到的数值，常见记法是 θ。可以把它们粗略想成模型内部可调整的旋钮。现代语言模型含有大量参数，但参数数量本身不说明模型一定正确、可靠或适合某任务。

模型接收输入后，中间会产生 **激活值（activations）** 。激活值是这次计算产生的临时数值；它们通常随输入变化。参数是模型保存下来的可学习数值。不要把“模型参数”和“本轮对话内容”混为一谈。

3. 预测、logits 和概率

对下一个 token，模型可以给词表中的每个候选项一个未归一化分数，叫 **logit**。Logit 不是概率：它可以为负数，不必在 0 与 1 之间，所有 logits 的和也不必为 1。

对 logits 做 softmax，可得到一组概率：每个概率在 0 和 1 之间，所有候选概率之和为 1。例如词表简化为 `{伞, 书, 水}`，模型输出概率 `{0.80, 0.15, 0.05}`，表示在当前模型和解码设置下，“伞”比另外两个候选更可能被选中。它不表示模型已经理解现实，也不保证“伞”在现实中一定正确。

自回归语言模型（autoregressive language model）会重复这一步：把新选出的 token 接到上下文后面，再预测下一 token。真实生成还有停止符、采样策略、最大长度等规则，留到阶段 1 学。

4. 训练和推理是两种不同工作模式

训练（training）

训练时，系统提供样本和目标。模型先产生预测，再计算预测与目标之间的差异（损失），然后根据梯度调整一部分参数。重复许多次后，参数可能更适合训练目标。

```
训练样本 + 目标
        ↓
   模型预测
        ↓ 与目标比较
      损失
        ↓ 求梯度、更新参数
    新参数
```

训练不是把事实像文件一样直接存进模型。参数更新会影响模型行为，但不等于精确、可追溯的数据库写入。数据质量、目标、训练过程和评估方式都会影响结果。

推理（inference）

推理时，系统把已经训练好的参数固定下来，只用它们对当前输入做计算。生成答案时，通常按 token 逐步计算和选择。推理期间对话上下文会改变，但常规推理不会因此自动修改模型参数。

```
固定参数 + 当前输入
        ↓
     模型计算
        ↓
      输出
```

微调（fine-tuning） 是在已有模型基础上继续训练，不是普通推理的同义词。阶段 2 会比较监督微调和偏好优化。

5. 需要分开的几个概念

概念	简明含义	常见误解
token	tokenizer 切出的文本单位	一个 token 一定等于一个字或一个词
token ID	词表中某个 token 的整数编号	编号大小代表语义或重要程度
参数（parameter）	训练中可学习的模型数值	对话内容就是模型参数
激活值（activation）	某次前向计算产生的中间值	每次推理都会永久写入模型
logit	候选输出的原始分数	logit 本身已经是百分比
概率	归一化后的候选相对概率	概率最高就等于事实为真
损失（loss）	对某个训练目标的误差度量	一个低损失能代表所有能力都好
推理（inference）	用当前参数计算输出	模型在推理时必然继续学习
上下文（context）	当前计算可用的输入信息	上下文窗口就是永久记忆

6. 上下文不是永久记忆

上下文是本次请求中模型能读取的信息，例如系统提示、历史消息和本轮问题。**上下文窗口（context window）** 指模型一次处理的上下文容量边界，通常以 token 数量衡量。它不是无限的，也不等同于长期记忆。应用程序可以保存历史并在之后重新提供，但那属于系统的数据管理机制。

7. 自己检查

不看上文，口头回答：

1. tokenizer 为什么要把文本转换成 token ID？
2. 参数和激活值分别是什么？哪一个通常跨请求保存在模型权重里？
3. logit 与概率的区别是什么？
4. 训练与推理时，参数分别发生什么？
5. 为什么不能把上下文窗口叫作永久记忆？